CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 06/Unit 01: Operator Overloading

Course taught by:

Courtesy of some slides: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)

Unit Outline

1. Recap on Function Overloading and Constructors
2. Operator Overloading – Intro
3. Overloading Binary Operators
4. Overloading Unary Operators
5. Assignment and Comparison
6. Overloading the extraction and insertion operators.
7. Summary

Function Overloading– Recap

- We learnt that **Function overloading** is a programming concept that allows multiple functions to have the same name but differ in the **number** or **type of their parameters**.
- Why need it? It enables programmers to create multiple versions of a function tailored to different input types or use cases, while maintaining intuitive naming.
- A function is said to be **overloaded** if an implementation of it has the same name but different signature.
- The signature of a function is the **number**, **type** and **order of its parameters**.

Overloading Operators – Introduction

- We just saw Overloaded functions are distinguished by the number, type or order of the function parameters – also called the function signature.
- C++ also allows for **operator** overloading; slightly different concept than function overloading!
- What this means is – you can assign a **different operation** for a **common operator** such as +, -, *, /, ++, >, <, etc.
- These additional definitions are useful for user-defined data types (e.g. complex numbers, vectors or matrices)

Overloading Operators – Motivation?

- Make user-defined types behave like built-in types

```
Distance d1(5, 6.5), d2(3, 4.5);
Distance d3 = add(d1, d2);     // clunky function call

Distance d3 = d1 + d2;         // natural, intuitive
```

Overloading Operators – Motivation?

- Improve readability and maintainability (Think of complex numbers and Matrices)

```
c3 = addComplex(c1, c2);
m3 = multiplyMatrix(m1, m2);
```

```
c3 = c1 + c2;       // clear
m3 = m1 * m2;       // closer to math notation
```

Overloading Operators – Motivation?

- Consistency with built-in types:
- Built-in types already support operators (+, -, *, /, ==, etc.).
- Operator overloading lets user-defined types join the same “family.”

Operator Overloading

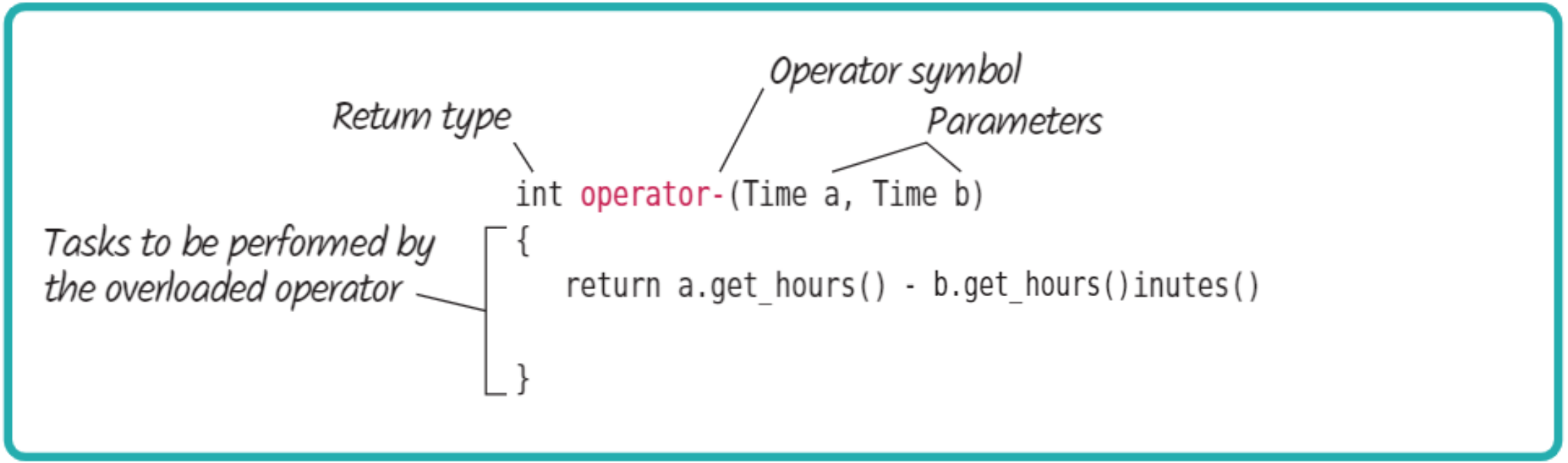
- Consider the Distance class example →
- How about adding two distance objects to produce a third distance object?
- In other words, how about d3 = d1 + d2? [Think: How will this work?]
  - One way is to write an **add_dist** function but that's not too **intuitive**, right?

```
class Distance {
private:
    int feet;
    float inches;

public:
    Distance(int ft = 0, float in = 0) {
        feet = ft;
        inches = in;
    }
};
```

Operator Overloading to the rescue!

- The keyword **`operator`** is used to **overload** an operator.
- This is used in a function declaration as follows:



Overloading Binary Operators

How to overload operators such as +, -, = etc.?

Overloading Binary Operators

- Binary operators (like +, -, *, /, >, <) work on two operands.
- To overload them for user-defined types, you define a function named operator<symbol>.
- There are two common ways to write such functions:
  - Define using both arguments:

```
Distance operator+(Distance d1, Distance d2)
```

called as: **`d3 = d1 + d2;`**

  - Define using the second argument (first argument is implicit):

```
Distance operator+(Distance d2)
```

called as: **`d3 = d1 + d2;`**

- Note that **`d1`** is the implicit parameter in the second type of definition

Overloading Binary Operators (+ Operator)

```
    // Declaration only inside main function
    Distance operator+(Distance d2) const;
```

```
// Definition using scope resolution operator
Distance Distance::operator+(Distance d2) const {
    int f = feet + d2.feet;             // left operand is implicit (*this)
    float i = inches + d2.inches;       // right operand is passed explicitly

    if(i >= 12.0) {
        i -= 12.0;
        f++;
    }
    return Distance(f, i);
}
```

Overloading Binary Operators (+ Operator)

```
int main() {
    Distance d1(5, 9.5), d2(3, 4.5);
    Distance d3 = d1 + d2;    // calls d1.operator+(d2)
    d3.display();             // Output: 9 feet 2 inches
    return 0;
}
```

- Q. Can you also do **Distance d4 = d1 + d2 + d3**?

Overloading Binary Operators (+ Operator)

```
int main() {
    Distance d1(5, 9.5), d2(3, 4.5);
    Distance d3 = d1 + d2;    // calls d1.operator+(d2)
    d3.display();             // Output: 9 feet 2 inches
    return 0;
}
```

- Q. Can you also do **Distance d4 = d1 + d2 + d3**?
- **A. Yes,** under the hood it will do something like this:

```
Distance temp1 = d1.operator+(d2);
Distance d4    = temp1.operator+(d3);
```

Overloading Binary Operators (Some rules)

- **Almost any operator that exists, can be overloaded.**
  - You may go ahead and overload +, -, *, /, +=, >, <, etc
  - This allows user-defined types (like Distance, Complex, Matrix) to behave like built-in types.
- **You must only overload operators that exists.**
  - Cannot invent a new operator symbol (e.g., ** for power).
  - Can redefine what + means for your class, but you cannot create brand-new operators.
- **At least one of the arguments in an overloaded operator must be a user defined type.**
  - Distance d1 is an example of a user defined data-type

Overloading Binary Operators (Some rules)

- **It is not possible to change the number of operands that an operator supports.**
  - + is binary → always takes 2 operands.
  - ++ is unary → always takes 1 operand.
  - You cannot overload + to suddenly take 3 arguments.
- **The precedence rules (associativity, precedence) between operators cannot be changed.**
  - Example:
  - a + b * c; // * will always bind before +, even if overloaded
- Some operators **cannot be overloaded** at all
  - :: (scope resolution), . (member access), .*, sizeof

Example: A Time Class

- Given two objects for the class **Time**, we are interested in overloading operators for these objects.
- We will overload **+, -, ==, !=, <**
- Let’s move to VS Code!

Overloading Unary Operators

How to overload operators such as ++ and --?

Operator Overloading – Unary Operators

- Unary operators act on one operand (examples: ++a, a++, --a, a--).
- Can be overloaded for user-defined classes.
- Just like binary operators, the compiler **decides** which function to call based on **operand type**.

```
int x = 5; ++x;
  // built-in int increment
Counter c1(); ++c1;
  // calls user-defined operator++
```

Overloading the ++ Operator (General Rule!)

Form	Syntax	Operator Function	How the compiler recognizes!
Prefix	`++c1`	`Counter operator++();`	No arguments
Postfix	`c1++`	`Counter operator++(int);`	Takes a **dummy int** argument

Takeaway: Prefix has no arguments, postfix has a **dummy int** to mark it as postfix.

Overloading the ++ Operator (Example!)

- Prefix Increment (`++t1`)
- Consider the class `Counter`:

```
class Counter {
public:
    int count;
    Counter() : count(0) {}
    Counter(int c) : count(c) {}

    // Prefix ++
    Counter operator++() {
        ++count;                // increment
        return Counter(count); // return updated value
    }
};
```

```
int main(){

    Counter c1(3);
    Counter c2 = ++c1;
    cout << c2.count <<endl;
}
```

Q. What is the output?

Overloading the ++ Operator (Example!)

- Prefix Increment (`++t1`)
- Consider the class `Counter`:

```
class Counter {
public:
    int count;
    Counter() : count(0) {}
    Counter(int c) : count(c) {}

    // Prefix ++
    Counter operator++() {
        ++count;                // increment
        return Counter(count); // return updated value
    }
};
```

```
int main(){

    Counter c1(3);
    Counter c2 = ++c1;
    cout << c2.count <<endl;
}
```

Q. What is the output?
A. 4

Key Point: Prefix increments first, then returns the **new value**.

Overloading the ++ Operator (Postfix!)

- Postfix Increment (c1++)
- Consider the class `Counter`:

```
class Counter {
public:
    int count;
    Counter() : count(0) {}
    Counter(int c) : count(c) {}

    // Postfix ++
    Counter operator++(int) {
        return Counter(count++); // return old value, then increment
    }
};
```

```
int main(){

    Counter c1(3);
    Counter c2 = c1++;
    cout << c2.count <<endl;
}
```

Q. What is the output?

Overloading the ++ Operator (Postfix!)

- Postfix Increment (`c1++`)
- Consider the class `Counter`:

```
class Counter {
public:
    int count;
    Counter() : count(0) {}
    Counter(int c) : count(c) {}

    // Postfix ++
    Counter operator++(int) {
        return Counter(count++); // return old value, then increment
    }
};
```

```
int main(){

    Counter c1(3);
    Counter c2 = c1++;
    cout << c2.count <<endl;
}
```

Q. What is the output?
A. 3

Key Point: Postfix returns the **old value**, then increments.

Overloading the ++ Operator (Summary!)

Expression	Action	Returned Value
`++c1`	Increment first	New value
`c1++`	Return copy → then increment	Old value

Decrement Operators

- Does the same mechanism applicable to increment operators also work on the decrement (--) operators.
- Home Exercise: Can you write a program to do pre (++m) post increment (m++) and pre (--m) and post-decrement (m--) for the Time Class.
  - Only do the increment, decrement on minutes!

Operator Overloading General Rule

- In a **operator function**, the **left operand** is the object that calls the function. Remember the example of add Operator+ from earlier slide.
- General Rule:
  - An overloaded operator requires **one fewer argument** than the number of operands, because one operand is the calling object.
- Example: Binary operator (+):
  - Distance operator+(Distance d2);
  - Accepts one parameter (d2), left operand (d1) is implicit
- Example: Unary Operator (++):
  - Counter operator++();
  - No parameters, the single operand is implicit

Overloading Extraction and Insertion Operators

How to overload operators such as >> and <<?

Overloading the extraction and insertion operators - << and >> operators

- To overload the **insertion (<<)** and **extraction (>>)** operators, we first need to understand how **input and output streams** work.
  - **cout** and **cin** are objects of **ostream** and **istream** Classes
  - These operators are already defined for basic types (e.g., **int**, **double**).
  - We can extend them to handle **user-defined classes**.
- Summary: **ostream** and **istream** are classes. They are defined in the <iostream> header.
- For detailed reading, please go through Chapter 8 of Cay Horstmann's book for the same.
  - Pdf attached on Canvas!

Overloading the extraction and insertion operators - << and >> operators

```
// Insertion operator (output)
ostream& operator<<(ostream& out, const Distance& d) {
    out << d.feet << " feet " << d.inches << " inches";
    return out;   // return stream for chaining
}


int main(){

    Distance d1;
    cout << d1 <<endl;

}
```

- Here **`ostream`** represents and output stream class
- This function accepts an output stream object as a parameter, along with a `Distance` object to be sent to the output stream
- It then returns the output stream by reference with the Distance object printed.

Overloading the extraction and insertion operators - << and >> operators

```
// Insertion operator (output)
ostream& operator<<(ostream& out, const Distance& d) {
    out << d.feet << " feet " << d.inches << " inches";
    return out;   // return stream for chaining
}


int main(){

    Distance d1;
    cout << d1 <<endl;

}
```

- When you call:

```
cout << d1 << endl;
```

- It really means:

```
(cout << d1) << "\n";
```

- Which calls

```
operator<<(cout, d1) <<"\n";
```

Overloading the extraction and insertion operators - << and >> operators

```
// Extraction operator (input)
istream& operator>>(istream& in, Distance& d) {
    in >> d.feet >> d.inches;
    return in;    // return stream for chaining
}


int main(){

    Distance d1;
    cin >> d1 <<endl;

}
```

- Same can be written for taking inputs for a class as well i.e. by overloading **instream** class

Rule of 3

- In C++, the Rule of 3 is a convention (best practice). It states that:
- If a class needs to define one of the following:
  - Destructor,
  - Copy Constructor
  - Copy Assignment Operator (**We just learnt this in the class today i.e. overloading = operator**)

then it needs to explicitly define all 3 in order to have exception safe coding.

Summary

- Operator overloading is a useful feature in C++.
- To overload an operator, we use the operator keyword followed by the operator.
- There are certain rules and restrictions applicable to operator overloading.